

Authentication with Auth.php

Checking Who a User Is

Picking Up From Session Management

Last time, `SessionAuth` handed out the wristband and checked it.

- `SessionAuth` never asked "is this password correct?"
- Session is required because HTTP is like a new cashier every time — it does not remember who you were before.

But the `login.php` you ran still checked a password — by comparing plain text.

- That was fine for learning sessions, but it is not secure authentication.
- Today we build the real door-checker that does this safely: `Auth`.
- `Auth` checks the password with hashing, and `SessionAuth` remembers the visitor afterward.

Let's discuss these ideas one by one.

Authentication vs. Authorization

Think of a clubhouse with a locked door and a secret password.

- **Authentication** is the password check at the door — "Are you really you?"
- **Authorization** is the sign inside that says who may touch the cookie jar.

Why Authentication Matters

Imagine a house with no lock on the front door. Anyone walking by could just open it and take anything inside.

```
// Without authentication – anyone can call this
if ($_GET['action'] == 'delete_user') {
    delete_user($_GET['id']); // Disaster!
}
```

A lock on the door means only someone who is checked in gets in.

- Here `SessionAuth`, the class from last lecture, is the lock check.

```
// With authentication – only a logged-in visitor can
$session = new SessionAuth();
if ($session->is_logged_in()) {
    delete_user($_GET['id']);
} else {
    echo "Permission denied";
}
```

Password is Replaced with Hashed Number

| What Is a Hash?

Picture a blender.

- You drop a banana in, and out comes banana smoothie.
- You can never turn that smoothie back into a whole banana.

A **hash** works the same way: data goes in, a scrambled code comes out.

The same banana always makes the exact same smoothie.

- A tiny change to the fruit makes a totally different smoothie.
- And nobody can "unblend" it back to the original.

```
Input:  Hello World
```

```
SHA-256: a591a6d40bf420404a011733cfb7b190  
          d62c65bf0bcda32b57b277d9ad9f146e
```

Never Store Plain Passwords

Never write someone's actual password on a sticky note for everyone to see.

- Store the "smoothie" (the hash) instead of the "banana" (the real password).

```
// WRONG – password is visible in the database  
$stmt->execute([$username, $password]);
```

```
// CORRECT – store only a hash  
$hash = password_hash($password, PASSWORD_DEFAULT);  
$stmt->execute([$username, $hash]);
```

To check a login, we don't "un-blend" the old smoothie.

- We blend the given password and see if the two smoothies match.

```
// Checking a login later – never decrypt, always re-hash and compare
if (password_verify($entered_password, $stored_hash)) {
    // correct password
}
```

`password_hash()` and `password_verify()` are the blender for you.

- `password_hash()` automatically generates a unique random **salt** each time — so hashing the same password twice produces different hash strings!
- `password_verify()` reads the salt and algorithm stored inside `$stored_hash` to verify the entered password.
- Never build your own blender — always use these two built-in functions.

The Basic Flow

Think of joining a treehouse club.

1. **Register** — you pick a password, and it gets turned into a smoothie and saved.
2. **Login** — you say the password again, we blend it, and check the smoothie matches.
3. **Protected access** — you wear a wristband so the club doesn't ask again every minute.
4. **Logout** — you take the wristband off when you leave.

| Not Authorization — Just Staying Checked-In

- `Auth` handles step 1 and 2.
- `SessionAuth` from last lecture handles step 3 and 4.

Authorization is a separate problem from authentication and requires its own rules and checks.

Step 3, "protected access," only answers one question: are you logged in at all?

- It does not ask what role you have, or what you are allowed to do.
- That single yes/no gate is still Authentication — proving who you are, request after request.

Notice that checking `$session->is_logged_in()` only checks *if* someone is logged in.

- What if only an admin should delete users (`role === 'admin'`)?
- That check needs a `role` field stored on the user.
- Neither `Auth` nor `SessionAuth` here stores a `role` — this lecture builds Authentication, not Authorization.
- Real Authorization (admin vs. member vs. guest) is a job for later, on top of what we build

A Minimal Working Example

`basic_auth.php` shows the whole treehouse flow in one small file.

It looks like the manual session pattern from last lecture, now with hashed passwords.

```
<?php
session_start();

$users = [
    'john' => password_hash('secret123', PASSWORD_DEFAULT),
    'jane' => password_hash('mypassword', PASSWORD_DEFAULT)
];
```

Login — check the password, then hand out the wristband.

```
if (isset($_POST['action']) && $_POST['action'] == 'login') {  
    $username = $_POST['username'] ?? '';  
    $password = $_POST['password'] ?? '';  
  
    if (isset($users[$username]) &&  
        password_verify($password, $users[$username])) {  
  
        $_SESSION['logged_in'] = true;  
        $_SESSION['username'] = $username;  
        echo "Welcome, $username!";  
    } else {  
        echo "Invalid credentials!";  
    }  
}
```

Check the wristband / take it off (logout):

```
// Handle logout before sending any output (prevents "headers already sent")
if (isset($_GET['action']) && $_GET['action'] == 'logout') {
    session_destroy();
    header('Location: basic_auth.php');
    exit;
}

// Display protected content or login form
if (isset($_SESSION['logged_in']) && $_SESSION['logged_in']) {
    echo "Hello, " . $_SESSION['username'] . "!";
    echo ' <a href="?action=logout">Logout</a>';
} else {
    // show login form
}
```

This works for a tiny demo with only two friends in the list.

A real club with many members needs a proper helper — that's **Auth** next.

A Minimal Auth Class

Think of **Auth** as the one club helper who checks everyone at the door.

```
class Auth {
    private $users_db;

    public function __construct($users_file = 'data/users.json') {
        $this->users_db = new JsonDatabase($users_file);
    }

    public function register($username, $password) {
        if ($this->find_user_by_username($username)) {
            throw new Exception('Username already exists');
        }
        $user = [
            'username' => $username,
            'password_hash' => password_hash($password, PASSWORD_DEFAULT),
        ];
        return $this->users_db->add($user);
    }
}
```

```
public function login($username, $password) {  
    $user = $this->find_user_by_username($username);  
    if (!$user || !password_verify($password, $user['password_hash'])) {  
        throw new Exception('Invalid credentials');  
    }  
    return $user;  
}  
  
public function find_user_by_username($username) {  
    return $this->users_db->find_by_field('username', $username);  
}  
}
```

JsonDatabase is the same notebook-style storage you have used before.

The users file is just a list of records with **add()** and **find_by_field()**.

Auth + SessionAuth, Together

Remember what `SessionAuth` asked for last lecture:

"Login code must verify the password first, then call `login_user($user)`."

`Auth` is that login code.

```
// login.php – Auth checks the password, SessionAuth hands out the wristband
$auth = new Auth();
$session = new SessionAuth();

try {
    $user = $auth->login($username, $password);
    $session->login_user($user);
    echo "Welcome, " . $user['username'] . "!";
} catch (Exception $e) {
    echo "Invalid credentials!";
}
```

Two small classes, two small jobs: **Auth** checks who you are, **SessionAuth** remembers you.

IMPORTANT

Why a Class Instead of Loose Functions?

Imagine ten different helpers, each with their own rulebook for checking passwords: If the rule changes, you must fix ten rulebooks instead of one.

	Scattered functions	Auth class
Update a rule	Edit every file	Edit one method
Consistency	Easy to diverge	Same rule everywhere
Reusability	Code repeated	Code reused via methods
Testing	Hard	Test the class alone
Extensibility	Hard to add features	Easy to extend

One helper with one rulebook means one source of truth for everyone.

Hands-On

The full `Auth.php` in the repository does even more.

- It handles login-attempt limiting, profile updates, and user statistics too.
- Read it once `register()` and `login()` above feel comfortable.
- Combine it with `SessionAuth` from the previous lecture to build a complete login flow.

| Run the example

Start the PHP server in

`7_Security/Basics/authentication`.

Access the server with

`http://localhost:8080/basic_auth.php`.

Watch Out: Asking for a Badge That Was Never Handed Out

Notice the `isset($_POST['action']) &&` guard in the login check a few slides back — it isn't decoration.

```
// Without the guard  
if ($_POST['action'] == 'login') {
```

- On the very first visit, nobody has submitted the form yet.
- `$_POST` is just an empty array — the `'action'` key was never handed out.

- Asking for a badge that was never given out doesn't crash the program, but PHP still complains.

Warning: Undefined array key "action" in ...

- This warning is PHP saying "you assumed this key exists, but I never received it."
- Left unfixed, warnings pile up in logs — and can leak file paths in error output, a small security smell of its own.

The Fix: Ask "Does It Exist?" First

```
// With the guard – this is what the code above actually does
if (isset($_POST['action']) && $_POST['action'] == 'login') {
    // ...
}
```

- `isset()` checks the key exists before anything else runs.
- `&&` short-circuits — if `isset()` is false, PHP never touches `$_POST['action']` at all.

The same fix belongs on the logout check further up in this same file:

```
// Before
if ($_GET['action'] == 'logout') { ... }

// After
if (isset($_GET['action']) && $_GET['action'] == 'logout') { ... }
```

	Unguarded	Guarded with <code>isset()</code>
First page visit	Warning in logs	Silent, safe
Missing form field	Warning in logs	Silent, safe
Cost	None	One extra check

This isn't hashing or sessions — it's a smaller habit underneath both: never trust that user input arrived the way you expected.

Key Takeaways

1. Authentication asks "who are you?"; authorization asks "what can you do?"
2. A hash is like a blender — you can never turn the smoothie back into the fruit.
3. Always hash passwords with `password_hash()` / `password_verify()`.
4. Never store plain passwords, and never write your own hashing.
5. `register()` and `login()` are the two core operations of an Auth class.
6. `Auth` checks the password; `SessionAuth` remembers the visitor afterward.